

Evolving LightRAG — Detailed Architecture

이 문서는 [README.md](#)에서 추상화한 세 카테고리(쿼리분석 및 전략선택·유저응답 / 로깅 및 로그분석 / 그래프 개선)의 세부 전략, 임계값, 지표, 이론적 배경, 한계를 다룹니다. 큰 그림과 에이전트 플로우는 README.md를 먼저 읽으세요.

목차

- [핵심 지표](#)
- [1. 쿼리분석 및 전략선택, 유저응답](#)
- [2. 로깅 및 로그분석](#)
- [3. 그래프 개선](#)
- [태깅: keywords vs source_id](#)
- [이론적 배경](#)
- [벤치마크](#)
- [한계](#)
- [TODO](#)
- [References](#)

핵심 지표

weight (엣지 가중치) — LightRAG 기본 제공, 3. 그래프 개선이 부여

그래프의 모든 릴레이션에 붙어있는 숫자로, 해당 관계의 확실한 정도를 나타냅니다. LightRAG가 같은 관계를 여러 문서에서 추출하면 자동으로 합산합니다(`operate.py` 의 `_merge_edges_then_upsert`). 검색 시 `(rank, weight)` 기준으로 내림차순 정렬됩니다.

출처	weight
원본 문서에서 LLM 추출	1.0 (여러 문서에서 재추출되면 누적)
그래프 개선 — 로그 기반 추론(co-retrieval/gap-fill/shortcut)이 생성	0.5 (<code>inferred_edge_weight</code>)

출처	weight
그래프 개선 — 코퍼스 메타데이터 기반 구조 마이닝이 생성	0.3 (<code>structural_edge_weight</code>)

구조 마이닝 엣지의 weight를 로그 기반 추론 엣지보다 낮게 잡은 이유: 구조 관계는 항상 참(문서가 실제로 그 엔티티를 포함하는 것은 사실)이지만, 의미적으로는 로그 기반 추론보다 느슨한 신호이기 때문입니다.

quality (쿼리 품질 점수) — 2. 로깅 및 로그분석이 산출, 0 LLM 호출

이번 턴에 검색된 서브쿼리 결과마다 산출하는 0.0~1.0 점수입니다.

결과	quality
엔티티 0 + 청크 0	0.0
엔티티 있으나 관계 없음	0.3
청크만 있고 엔티티 없음	0.4
엔티티 + 관계 + 청크	0.5~1.0 (수량에 비례)

quality는 더 이상 그래프 개선 실행 여부를 게이팅하지 않습니다 — tier 1은 지연 없이 백그라운드에서 도니 매 쿼리 실행해도 비용이 문제되지 않기 때문입니다. 대신 gap filling(엔티티 0, 청크>0)처럼 quality 자체보다 더 구체적인 조건으로 트리거되는 전략들이 있고, quality는 로그 분석·모니터링용 지표로 남습니다.

access_count / last_accessed — 2. 로깅 및 로그분석이 갱신

쿼리마다 검색된 엔티티의 `access_count` 를 1 증가시키고 `last_accessed` 를 갱신합니다. 린팅의 방치 엔티티 탐지, 그래프 개선이 만든 추론 엣지의 자동 정리 조건으로 쓰입니다.

full_doc_id / chunk_order_index — LightRAG 기본 제공, 3. 그래프 개선이 활용

모든 청크(`text_chunks`)는 원본 문서 ID(`full_doc_id`)와 문서 내 순서(`chunk_order_index`)를 갖습니다. 그래프 개선은 이 필드만으로 문서 노드와 청크 인접 관계를 만듭니다 — LLM 호출이 전혀 필요 없습니다.

1. 쿼리분석 및 전략선택, 유저응답

wikigraph/operations/analyze.py (ANALYZE), wikigraph/operations/query.py 의 respond_node (RESPOND) — 유일하게 즉시(inline) 실행되는 단계입니다.

ANALYZE

LLM에게 한 번의 호출로 두 가지를 동시에 요청합니다:

1. 재작성(rewrite): 모호함 해소, 축약어 확장, 대화체 제거 — 검색에 최적화된 단일 쿼리로 변환 (`optimized_query`)
2. 분해(decompose): 질문이 여러 독립적인 정보 요구를 담고 있으면 최대 `query_decompose_max_subqueries` (기본 3)개의 서브쿼리로 분리 (`sub_queries`). 단일 요구면 재작성된 쿼리 하나만 반환합니다.
3. 문서 언급 감지: 질문이 특정 문서/파일을 제목·파일명·명확한 지칭("Q3 보고서" 같은)으로 언급하면 콤마로 나열하고, 아니면 `NONE` (`mentioned_docs`)

파싱은 `OPTIMIZED: / SUB1: , SUB2: , ... / DOCS:` 형식의 정규화된 응답을 기대하며, 파싱에 실패하거나 `llm_func` 가 없으면 원본 쿼리를 그대로 통과시키고 `mentioned_docs` 는 빈 리스트로 둡니다 (fail-safe).

전략 선택

ANALYZE의 출력은 세 곳에 동시에 쓰입니다 — 하나만 골라 실행하는 게 아니라 전부 실행됩니다:

- `optimized_query` → RESPOND로 즉시 전달
- `sub_queries` → 백그라운드 큐에 등록되어 [2. 로깅 및 로그분석](#)의 RETRIEVE 단계로 전달
- `mentioned_docs` → RESPOND의 문서 스코핑 판단 근거로 전달 (아래)

이렇게 해야 co-retrieval, gap filling 같은 그래프 개선 전략이 "이 정보 요구에 대해 무엇이 검색됐는가"를 서브쿼리 단위로 정확히 볼 수 있습니다.

RESPOND (유저응답)

`optimized_query` (또는 ANALYZE 결과가 없으면 원본 쿼리)로 `rag.aquery(..., mode="mix")` 를 호출해 답을 합성하고 즉시 리턴합니다. 그래프 개선이 끝나길 기다리지 않습니다 — 이번 답변에는 아직 반영되지 않은 개선이라도, 다음 쿼리부터는 반영됩니다.

문서 스코핑: `mentioned_docs` 가 있으면, `wikigraph/operations/query.py` 의 `_match_document_nodes()` 가 그래프에서 3단계 구조 마이닝이 이미 만들어 둔 `doc::` 노드 중 이름/description이 일치하는 걸 찾고, `_scoped_entities()` 가 그 노드에 `CONTAINS` 로 연결된 엔티티를

최대 `doc_scope_max_entities` (기본 30)개 모읍니다. 매칭된 엔티티가 있으면 `QueryParam(mode="mix", ll_keywords=entities)` 로 검색합니다.

이게 왜 되는가: LightRAG의 `kg_query (operate.py)` 는 `hl_keywords / ll_keywords` 가 이미 채워져 있으면 자체 LLM 키워드 추출 단계를 건너뛰고 그 값을 그대로 씁니다. `ll_keywords` 는 `local / hybrid / mix` 모드에서 엔티티 벡터 검색에 쓰이므로, 매칭된 문서의 엔티티만 넘기면 검색이 그 문서 주변으로 좁혀집니다.

의도적으로 RETRIEVE에는 적용하지 않음: 문서 스코핑은 RESPOND에만 적용되고, [2. 로깅 및 로그분석](#)의 RETRIEVE에는 적용되지 않습니다. RETRIEVE의 역할은 그래프 개선이 패턴(co-retrieval, gap, shortcut)을 찾을 수 있도록 넓게 탐색하는 것인데, 스코핑을 걸면 바로 그 발견 대상인 교차 문서 신호를 가려버리기 때문입니다.

fail-safe: 문서 언급과 일치하는 `doc::` 노드가 없으면(3단계 구조 마이닝이 아직 그 문서를 못 봤거나 — 배치 주기라 신선도가 안 맞을 수 있음 — 애초에 매칭이 안 되면) 조용히 스코핑 없이 평소처럼 검색합니다.

2. 로깅 및 로그분석

`wikigraph/operations/query.py` 의 `retrieve_node / evaluate_node` , `wikigraph/metadata.py` — 백그라운드 파이프라인의 앞단이며, 3단계 그래프 개선이 참조하는 모든 근거가 여기서 만들어집니다.

RETRIEVE

`sub_queries` 각각을 독립적으로 `rag.aquery_data()` 로 리트리브해 엔티티/관계/청크를 뽑아내고, `QueryLogEntry` 로 `query_log` 에 쌓습니다. 동시에 검색된 각 엔티티의 `entity_metadata (access_count , last_accessed)` 를 갱신합니다.

EVALUATE

이번 턴에 새로 쌓인 로그 항목마다 `_score_quality(entities, relations, chunks)` 규칙으로 quality를 매기고, 평균이 `quality_evolve_threshold` 미만이면 `should_evolve` 플래그를 세웁니다 — 다만 앞서 언급했듯 이 플래그가 그래프 개선 실행 자체를 막지는 않고, 로그 분석·모니터링 신호로 남습니다.

로그 분석

`query_log` 는 단순 축적이 아니라, 3단계 그래프 개선이 곧바로 스캔할 수 있는 분석 대상입니다:

- co-retrieval 카운트: 같은 턴에 함께 검색된 엔티티 쌍이 로그 전체에서 몇 번 반복되는지
- gap 탐지: 청크는 검색됐는데 엔티티가 비어 있는 로그 항목 (추출 누락 신호)
- 반복 경로 탐지: 로그에 쌓인 릴레이션에서 $A \rightarrow B \rightarrow C$ 같은 패턴이 몇 번 나타나는지

이 세 가지 분석 결과가 각각 그래프 개선의 co-retrieval 강화, gap filling, shortcut path 전략을 트리거하는 입력이 됩니다.

3. 그래프 개선

`wikigraph/operations/evolve.py` — 2단계가 쌓은 로그-리트리브 결과와 코퍼스 메타데이터를 근거로 그래프에 실제 변경을 가하는 단계입니다. 근거가 쿼리 로그 쪽이든 코퍼스 메타데이터 쪽이든 나누지 않고 하나의 카테고리, 하나의 배치 파이프라인으로 취급합니다 — 로그 기반 뮤테이션과 구조 마이닝은 코드도 이 한 모듈에 함께 있습니다(예전엔 `structural.py` 로 분리돼 있었지만, 문서 구조를 병합하면서 코드도 병합했습니다).

Tier 1 — `evolve_light_node` , 매 쿼리마다 (백그라운드)

가볍고 증분적입니다. 이번까지 누적된 쿼리 로그(`query_log` , 최대 `max_query_log_size` 개)를 보고 세 가지 전략을 실행합니다.

co-retrieval 강화: 쿼리 로그에서 `co_retrieval_min_count` (기본 3)회 이상 함께 검색된 엔티티 쌍에 직접 연결이 없으면, LLM이 양쪽 description을 보고 관계를 추론하여 새 엣지를 생성합니다 (`llm_func` 없으면 "Frequently co-retrieved (N times)"라는 일반 설명으로 대체).

gap filling: 쿼리에서 원본 청크는 검색됐지만 그래프 엔티티가 없는 경우(추출 누락), 해당 청크 텍스트를 LLM에 전달해 엔티티/관계를 재추출합니다. 원본 청크에 근거가 있는 경우에만 동작하며, 이 전략만이 새 엔티티 노드를 추가할 수 있습니다. 트리거는 quality 임계값이 아니라 "청크는 있는데 엔티티가 없다"는 조건을 직접 확인합니다.

shortcut path: 쿼리 로그의 릴레이션에서 $A \rightarrow B \rightarrow C$ 패턴이 `shortcut_path_min_count` (기본 3)회 이상 나타나는데 $A \rightarrow C$ 직접 엣지가 없으면 생성합니다.

전체 예산은 `evolve_max_mutations` (기본 10)로 제한됩니다.

Tier 2 — `batch_evolve_node` , N번째 쿼리마다 (백그라운드)

`batch_evolve_interval` (기본 50)마다 트리거됩니다 (`should_run_batch_evolve` 로 판단 — 순수 함수라 단위테스트하기 쉽습니다). 세 가지를 추가로 합니다:

1. Tier 1과 같은 세 전략을 더 넓은 그물(임계값을 1 낮춤)과 더 큰 예산 (`batch_evolve_max_mutations` , 기본 30)으로 재실행 — 개별 쿼리 단위에서는 안 보이던 패턴이 누적된 배치에서는 보일 수 있습니다.
2. source verification (근거 상실 관계 제거): 그래프의 모든 (추출된) 엽지의 `source_id` 를 검사해 해당 청크가 `text_chunks` 에서 사라졌으면 제거합니다. 문서가 삭제/수정되면 그래프가 자동으로 따라갑니다. 그래프 개선이 직접 만든 엽지는 검사 대상에서 제외됩니다 ([태깅 절](#) 참고).
3. contradiction resolution (모순 통합): 같은 엔티티가 여러 문서에서 재등장해 `GRAPH_FIELD_SEP` 로 병합된 description이 2개 이상이면, LLM에게 하나로 통합하도록 요청합니다.

이 세 가지를 tier 2로 미룬 이유는 그래프 전체를 스캔해야 하기 때문입니다 — 매 쿼리마다 돌리면 그래프가 커질수록 백그라운드 큐가 밀립니다.

구조 마이닝 — tier 2와 같은 주기 (`evolve.py` 의 `structural_evolve_node`)

LightRAG의 엔티티/릴레이션 추출은 각 청크 안에서 LLM이 찾아낸 의미적 관계만 그래프에 남깁니다. 다음과 같은 구조적 관계는 아무것도 잡지 못합니다:

- 같은 문서에 속한 두 청크가 서로 이웃해 있다는 사실
- 서로 다른 문서 A, B가 같은 개념을 여러 번 언급한다는 사실 (문서 간 연관성)
- 어떤 엔티티가 어느 문서에서 왔는지 (역참조)

이 정보는 이미 LightRAG의 `text_chunks` 에 `full_doc_id` , `chunk_order_index` 로 존재하는데, 그래프에는 투영되지 않고 있었습니다. 구조 마이닝은 이걸 LLM 호출 없이 그래프 엽지로 명시화합니다:

A. 문서 노드 + CONTAINS: 각 `full_doc_id` 마다 `doc::{full_doc_id}` 형태의 pseudo-노드를 만들고 (`entity_type="document"`), 그 문서에서 추출된 모든 엔티티에 `document -[CONTAINS]-> entity` 엽지를 만듭니다.

B. 문서-문서 관계: 두 문서가 공유하는 엔티티 수가 `structural_doc_shared_entity_min` (기본 2) 이상이면 두 문서 노드 사이에 `shares_entities` 엽지를 만듭니다. 이것이 "문서 대 문서의 구조적 관계"에 대한 직접적인 답입니다.

C. 청크 인접 → 엔티티 NEAR: 청크 자체를 그래프 노드로 만들면 그래프가 급격히 커지므로, 대신 같은 문서에서 `chunk_order_index` 차이가 `structural_chunk_adjacency_max_gap` (기본 1) 이하인 인접 청크들에서 추출된 엔티티 쌍에 (아직 연결이 없다면) `chunk_adjacency` 엽지를 만듭니다. 청크 쌍당 최대 1개 엽지만 추가해 그래프 폭증을 막습니다.

세 전략 모두 `structural_max_mutations` (기본 20) 예산을 공유합니다.

실제로 도움이 되는가

이론적으로는 멀티홉 질의("이 두 문서에 공통으로 언급된 게 뭐야?" 같은)에서 그래프 순회만으로 답을 찾을 수 있게 해줍니다. 다만 [한계](#) 절에서 언급하듯, 실제 검색 품질 개선 여부는 아직 정량 검증되지 않았

습니다.

부가: 린팅 (수동 호출)

wikigraph/operations/lint.py — 그래프 알고리즘 기반, 대부분 0 토큰. agent.lint() 로 수동 호출하며 자동 파이프라인(tier 1/2)에는 포함되지 않습니다. 그래프 개선이 만든 결과물의 품질을 점검하는 보조 도구로 여기 묶어 둡니다.

체크	방법	자동 수정
고아 노드	<code>node_degree() == 0</code>	아니오 (리포트만)
허브 과부하	<code>node_degree() > hub_degree_threshold (50)</code>	아니오
중복 엔티티	이름 임베딩 코사인 유사도 > <code>duplicate_similarity_threshold (0.95)</code> , 공통 토큰으로 사전 필터링	아니오
방치 엔티티	<code>access_count < 2</code> 이고 <code>stale_query_threshold (20)</code> 쿼리 이상 미접근	아니오
추론 엣지 정리	<code>keywords</code> 에 <code>wikigraph_evolve</code> 포함 + <code>weight ≤ 0.5</code> + 양쪽 엔티티 <code>access_count</code> 모두 0	예 — 유일한 자동 수정

구조 마이닝 엣지(`wikigraph_structural`)는 절대 자동 정리 대상이 아닙니다 — 문서가 실제로 그 엔티티를 포함한다는 사실은 검색에 쓰였는지 여부와 무관하게 참이기 때문입니다. 오직 로그 기반 추론으로 만든 엣지만 "한 번도 안 쓰이면 지운다"는 규칙의 대상입니다.

태깅: keywords vs source_id

3단계 그래프 개선은 그래프에 엣지를 주입할 때 `rag.ainsert_custom_kg()` 를 씁니다. 이 API는 릴레이션의 `source_id` 필드를 "이 값과 같은 `source_id` 를 가진 청크를 같이 넘겼는지" 확인해서 실제 청크 ID로 치환합니다. 그래프 개선은 청크를 넘기지 않으므로 (`"chunks": []`), 이 조회는 항상 실패하고 저장되는 `source_id` 는 항상 `"UNKNOWN"` 이 됩니다 — 우리가 넣으려던 태그가 아닙니다.

반면 `keywords` 필드는 이런 치환 없이 그대로 저장됩니다. 그래서 이 프로젝트의 모든 주입 엣지는 출처를 `keywords` 에 표시합니다:

- 로그 기반 추론: `"wikigraph_evolve,co-retrieved"` , `"wikigraph_evolve,gap-fill"` , `"wikigraph_evolve,shortcut,via_X"`

- 구조 마이닝: "wikigraph_structural,contains" ,
"wikigraph_structural,shares_entities" , "wikigraph_structural,chunk_adjacency"

source verification(근거 상실 관계 제거)과 린팅의 추론 엣지 자동 정리는 모두 keywords 를 검사합니다 — source_id 로 판단했다면 모든 주입 엣지가 "UNKNOWN 소스"로 보여서, source verification이 다음 배치 사이클에 즉시 다 지워버렸을 것입니다. harness_pjt/tests/test_wikigraph.py 의 test_evolve_light_injects_co_retrieval_edge_tagged_via_keywords 가 이 동작을 실제 LightRAG 그래프 스토리지로 검증합니다.

이론적 배경

전략 1~3(co-retrieval, gap filling, shortcut)은 Knowledge Graph Completion(KGC) 분야의 기존 연구에 기반합니다.

Co-retrieval → Link Prediction: 엔티티 공동 출현(co-occurrence)으로 누락된 엣지를 예측하는 것은 KGC의 표준 접근법입니다. [NoGE](#)는 엔티티-릴레이션 간 공동 출현 빈도를 그래프 임베딩에 통합해 link prediction 성능을 개선합니다.

Gap Filling → Extraction Repair: [Self-Improving RAG for KG Construction](#)은 추출 누락을 피드백 루프로 보강하는 프레임워크를 제안합니다. "청크에 근거가 있는 경우에만 재추출"로 제한해 hallucination을 방지합니다.

Shortcut → Transitive Closure: $A \rightarrow B \rightarrow C$ 에서 $A \rightarrow C$ 를 추론하는 것은 transitive closure 기반 KGC의 기본 원리입니다. [SMORE](#)는 대규모 KG에서 multi-hop reasoning을, [Practical GraphRAG](#)는 그래프 순회와 벡터 검색의 hybrid retrieval을 제안합니다.

Structural Relations → Document/Chunk Graph: 문서 간·청크 간 구조를 그래프에 명시하는 접근은 계층적 인덱싱(hierarchical indexing) 연구와 맞닿아 있습니다 — 텍스트 유사도만으로는 놓치는 "같은 문서에서 왔다", "인접해 있다" 같은 조직적(organizational) 신호를 검색에 활용할 수 있게 합니다.

벤치마크

정답지(gold answer) 기준 정확도를 수동으로 관찰한 수치입니다 — 정식 데이터셋·재현 스크립트는 아직 없는 프로토타입 단계 측정치라, 정량적 엄밀함보다는 방향성 확인용으로 봐 주세요.

구성	정확도
vanilla LightRAG (그래프 개선 없음)	~69%

구성	정확도
Evolving LightRAG (그래프 개선 적용)	64% ~ 72% (실행마다 변동)

관찰: 그래프 개선이 항상 vanilla보다 나은 게 아닙니다. 잘 나올 땐 +3%p(72%)까지 오르지만, 나쁠 땐 vanilla보다 오히려 -5%p(64%)까지 떨어지는 경우도 관찰됐습니다.

원인 추정: 구조 마이닝이 만드는 문서-문서-청크-청크 엣지가 기존 지식그래프에 그대로 얹히면서, weight를 낮게 줬음에도(0.3) 검색 시 노이즈로 작용하는 것으로 보입니다. 문서 수가 늘수록 `CONTAINS` / `shares_entities` / `chunk_adjacency` 엣지 수가 원본 의미 관계 엣지 수를 압도하기 쉬워, mix 모드의 그래프 순회가 구조 엣지 쪽으로 새는 경우가 생기는 것 같습니다 — 아래 TODO의 "구조 그래프 분리"가 이 문제를 겨냥한 대응입니다.

한계

노이즈 누적 (실측됨): 그래프 개선이 추가한 관계가 반복적으로 쌓이면 그래프에 노이즈가 증가할 수 있습니다 ([RAG Survey, 2025](#)). 실제로 [벤치마크](#)에서 vanilla 대비 정확도가 개선(72%)되기도, 악화(64%)되기도 하는 걸 확인했습니다. `weight` 마킹 + 린팅 자동 정리(로그 기반 추론 엣지에 한함)로 대응하고 있지만 충분하지 않아 보이며, 구조 마이닝 엣지를 아예 별도 그래프로 분리하는 방향을 검토 중입니다(TODO 참고).

수확 체감: [Iterative GraphRAG 연구](#)에 따르면 반복 3회 이후 추가 이득이 급감합니다. `evolve_max_mutations` / `batch_evolve_max_mutations` 제한으로 대응하지만, 최적 주기 (`batch_evolve_interval`)는 도메인에 따라 튜닝이 필요합니다.

LLM 추출 정확도: 엔티티/관계 추출 정확도가 도메인에 따라 [60-85%](#) 수준입니다. gap filling이 재추출해도 같은 실수를 반복할 수 있습니다.

구조 마이닝이 오히려 노이즈원일 수 있음: 문서-문서/청크-청크 엣지가 멀티홉 질의를 돕는지 검증하려 했으나, [벤치마크](#) 결과 오히려 기존 지식그래프 검색을 방해하는 쪽으로 작용하는 정황이 보입니다. 단순히 그래프가 촘촘해지는 것과 검색 품질이 좋아지는 것은 다른 문제라는 게 이번에 구체적으로 드러난 셈입니다.

단일 워커 직렬화: 백그라운드 파이프라인은 단일 워커 코루틴이 큐를 순서대로 처리합니다(동시성 버그를 피하기 위한 설계, `agent.py` 참고). 이는 correctness를 위한 트레이드오프이며, 쿼리가 매우 빈번한 워크로드에서는 큐가 밀릴 수 있습니다.

동일 LLM Wiki 대비 부족한 점: 인간 가독성(위키 페이지처럼 사람이 읽고 검토), confidence decay(시간 경과에 따른 신뢰도 감소), human-in-the-loop 승인 단계는 아직 구현되지 않았습니다.

TODO

- ☐ 구조 그래프 분리: 문서-문서/청크-청크 구조 마이닝 결과를 기존 지식그래프에 직접 주입하지 않고 별도의 구조 그래프로 분리해서 구축·진화시키기. 검색 시에는 구조 그래프를 먼저 탐색해 후보 문서/청크로 범위를 좁힌 다음, 그 범위 안에서만 기존 지식그래프를 탐색하는 2단계 파이프라인으로 전환 — [벤치마크](#)에서 관찰된 노이즈 문제(72%→64%대 변동)에 대한 대응
 - ☐ vanilla LightRAG vs Evolving LightRAG 검색 품질 정량 비교 — 재현 가능한 데이터셋·스크립트로 정식화 (지금은 수동 관찰치뿐)
 - ☐ 구조 그래프 분리 후, 문서/청크 구조 관계가 멀티홉 질의에 실제로 도움이 되는지 재검증
 - ☐ 엔티티별 마크다운 요약 페이지 자동 생성 (인간 가독성 확보)
 - ☐ timestamp 기반 confidence decay
 - ☐ Human-in-the-loop: 그래프 개선 결과를 사용자가 승인/거부
 - ☐ 대규모 코퍼스(1000+ 문서)에서의 구조 마이닝 성능/그래프 크기 장기 검증
 - ☐ tier1/tier2/구조 마이닝 간 피드백 루프 안정성 검증
-

References

Architecture

- [Karpathy's LLM Wiki](#) (2026.04)
- [LLM Wiki v2](#)
- [LightRAG](#) (HKUDS, 2024)
- [What LLM Wiki Is Missing](#)

Knowledge Graph Evolution

- [On the Evolution of Knowledge Graphs: A Survey](#)
- [Iterative Retrieval in GraphRAG: Diminishing Returns](#)
- [RAG Comprehensive Survey: Noise and Robustness](#)

Strategy Foundations

- [NoGE: Node Co-occurrence based GNN for KG Link Prediction](#)
- [SMORE: KG Completion and Multi-hop Reasoning](#)
- [Practical GraphRAG: Hybrid Retrieval at Scale](#)
- [Self-Improving RAG for KG Construction](#)
- [Calibrated Fusion for Multi-Hop QA](#)